Nintendo 3DS en Jetpack Compose

Guia para estudiar el codigo de game.kt (nivel 2º DAM) - version actualizada

1. Que hace la app

Dibuja una consola Nintendo 3DS azul con dos pantallas, copiando una imagen de referencia: arriba una escena tipo Mario (cielo, colinas, nubes, bloques, monedas, tubo, Mario y HUD) y abajo la pantalla tactil con "WORLD 1.1", barra de progreso, monedas, Score y los botones SELECT / HOME / START. A los lados, joystick, cruceta, botones X Y A B y los interruptores de volumen y 3D. No hay logica de juego: es solo diseño de UI.

2. Idea central: un lienzo con proporcion fija

En la primera version las medidas eran dp sueltos y la consola se estiraba en tablet. Ahora todo se dibuja sobre un **lienzo de 746 x 720 "pixeles de referencia"** (las medidas se sacaron de la imagen). Ese lienzo se escala con **una sola unidad u** (dp por pixel de referencia):

```
val u = minOf(maxWidth / ANCHO, maxHeight / ALTO)   // ANCHO = 746, ALTO = 720
Box(Modifier.size(u * ANCHO, u * ALTO)) { ... }      // lienzo escalado
```

- Se coge el **minimo** entre la escala por ancho y por alto: asi la consola siempre cabe entera.
- Como ancho y alto se multiplican por la misma u, la proporcion no cambia: **no se deforma** en movil ni en tablet; sobra espacio oscuro a los lados o arriba/abajo y la consola queda centrada.
- Cada elemento se coloca con su posicion (x, y) y tamaño (w, h) en pixeles de referencia, multiplicados por u.

3. Conceptos clave

3.1 Jetpack Compose y @Composable

Interfaz moderna de Android escrita con funciones de Kotlin. Cada funcion que dibuja algo lleva @Composable.

3.2 Box, BoxWithConstraints y Modifier

Elemento	Que hace	Uso en el codigo
Box	Apila hijos unos encima de otros	Cada pieza del dibujo es un Box
BoxWithConstraints	Box que te dice su maxWidth / maxHeight	Para calcular la escala u
Modifier	Cambia aspecto/comportamiento (el orden importa)	offset, size, clip, background, border

- .offset(x, y) -> desplaza el elemento desde su esquina superior izquierda.
- .size(w, h) -> tamaño. dp es la unidad de Android; sp la del texto.
- .clip(forma) -> recorta (CircleShape, RoundedCornerShape, GenericShape...).
- .background(color) / .border(grosor, color, forma) -> relleno y borde.

3.3 Coordenadas relativas

Un Box sin alineacion coloca a sus hijos en su esquina superior izquierda. Por eso, si una Pieza contiene otra Pieza, las coordenadas del hijo son **relativas al padre** (ej.: la pantalla del juego esta en (6, 4) dentro del marco negro). Ademas, clip() recorta a los hijos para que no se salgan.

4. El codigo explicado por partes

4.1 Constantes y colores

```
private const val ANCHO = 746f
private const val ALTO = 720f
private val Azul = Color(0xFF0050A1)   // ARGB: FF = opaco, luego R G B
```

Los colores repetidos (Negro, Azul, Gris, Amarillo, Marron) van en constantes; los que se usan una vez se escriben directamente en su pieza. private = solo visible dentro del archivo.

4.2 Las tres ayudas: Pieza, Circulo y Texto

```
@Composable
private fun Pieza(u: Dp, x: Float, y: Float, w: Float, h: Float, color: Color,
                  forma: Shape = RectangleShape,
                  contenido: @Composable () -> Unit = {}) {
    Box(Modifier.offset(u * x, u * y).size(u * w, u * h)
        .clip(forma).background(color)) { contenido() }
}
```

- **Pieza**: un rectangulo con posicion, tamaño, color y forma opcional. Acepta un bloque de contenido (lambda) donde se dibujan sus hijos.
- **Circulo**: llama a Pieza con CircleShape; recibe el *centro* (cx, cy) y el diametro d.
- **Texto**: un Text con offset, fuente monoespaciada y tamaño (u * tam).sp, para que el texto tambien escale.
- forma = RectangleShape y contenido = {} son **parametros con valor por defecto**: si no los pasas, se usan.

Gracias a estas tres funciones, cada elemento de la consola es **una sola linea**, por ejemplo:

```
Pieza(u, 0f, 224f, 563f, 90f, Color(0xFF0B8A00))   // suelo de hierba
Circulo(u, 14f, 18f, 13f, Amarillo)                 // moneda del HUD
```

4.3 Game: la funcion principal

```
@Composable
fun Game(modifier: Modifier = Modifier) {
    BoxWithConstraints(modifier.fillMaxSize().background(Color(0xFF222222))
                       .systemBarsPadding(), contentAlignment = Alignment.Center) {
        val u = minOf(maxWidth / ANCHO, maxHeight / ALTO)
        Box(Modifier.size(u * ANCHO, u * ALTO)
            .clip(RoundedCornerShape(u * 30)).background(Azul)
            .border(u * 2, Negro, RoundedCornerShape(u * 30))) {
            Cabecera(u); PantallaSuperior(u); Bisagra(u)
            PantallaInferior(u); Controles(u)
        }
    }
}
```

- Es la unica funcion publica; MainActivity la llama dentro de setContent { Game() }.
- El fondo oscuro ocupa toda la pantalla; systemBarsPadding() evita solaparse con la barra de estado.
- El Box azul es la carcasa de la consola (esquinas redondeadas y borde negro).
- Dentro se llaman 5 funciones, una por zona. Las que se dibujan despues quedan encima de las anteriores.

4.4 Cabecera y Altavoz

Cabecera dibuja los tres circulos de arriba (dos camaras negras y el sensor blanco central) y los dos altavoces. **Altavoz** dibuja 9 puntos con forma de rombo a partir de una lista de desplazamientos:

```
val puntos = listOf(0 to -2, -1 to -1, 1 to -1, -2 to 0, 0 to 0, 2 to 0, -1 to 1, 1 to 1, 0 to 2)
puntos.forEach { (i, j) -> Circulo(u, cx + i * s, cy + j * s, 8f, Negro) }
```

En vez de escribir 9 circulos a mano, se recorre la lista con forEach (los pares to se desestructuran en (i, j)).

4.5 PantallaSuperior y Escena

PantallaSuperior es el marco negro con la pantalla de cielo dentro. **Escena** dibuja el juego en capas (orden = de fondo a frente):

- Nubes: rectangulos con CircleShape (se convierten en ovalos).
- Colinas: esquinas de arriba redondeadas con RoundedCornerShape(topStart, topEnd).
- Suelo: tres franjas (hierba oscura, hierba clara, tierra).
- Bloques marrones, tubo amarillo y bloques grandes de la derecha (marron y morado).
- Monedas en diagonal: una lista de pares (x, y) recorrida con forEach.
- **Mario**: tres piezas apiladas - gorra roja, cara color piel y cuerpo rojo.
- HUD: moneda amarilla + "49" a la izquierda y "490" a la derecha.

4.6 Bisagra

Dos franjas horizontales de distinto tono de azul entre las dos pantallas.

4.7 PantallaInferior

Marco negro con la pantalla tactil gris claro dentro. En ella: titulo "WORLD 1.1", circulo rojo con "x005", barra de progreso con una linea blanca de puntos, tres monedas grandes (una amarilla, dos grises), pestaña "Score" con su caja, y una caja grande a la derecha. Debajo, los tres botones:

```
listOf(144f to "SELECT", 298f to "HOME", 450f to "START")
    .forEachIndexed { i, (x, t) ->
        Pieza(u, x, 692f, ancho, 24f, Negro) {              // borde negro
            Pieza(u, 2f, 2f, ancho - 4f, 20f, Color(0xFF0A3F88))  // fondo azul
            Texto(u, ancho / 2 - t.length * 4.5f, 5f, t, 14f, Color(0xFFE0E0E0))
        }
    }
```

Truco del borde: una pieza negra con otra azul 2 px mas pequeña dentro deja un borde negro de 2 px. El texto se centra restando la mitad de su anchura estimada (t.length * 4.5).

4.8 Controles

- **Joystick**: circulo negro con otro gris mas pequeño encima.
- **Cruceta**: dos rectangulos negros en cruz + cuatro lineas blancas finas (una por brazo).
- **Botones X, Y, A, B**: lista de (letra, posicion) recorrida con forEach; cada uno es un circulo negro con letra blanca.
- **Interruptores VOL y 3D**: franja azul, pieza gris, triangulo negro y textos VOL/OFF y 3D/OFF.

```
val trianguloIzq = GenericShape { s, _ ->
    moveTo(0f, 0f); lineTo(s.width, 0f); lineTo(0f, s.height); close()
}
```

GenericShape permite crear una forma propia dibujando un camino (path) punto a punto; s es el tamaño de la pieza.

4.9 Preview

```
@Preview(showBackground = true, widthDp = 800, heightDp = 1280)
@Composable
private fun GamePreview() { _3DSTheme { Game() } }
```

Permite ver el diseño en Android Studio sin ejecutar la app. Se usa un tamaño tipo tablet para comprobar que no se estira.

5. Conexion con MainActivity

```
class MainActivity : ComponentActivity() {
    override fun onCreate(savedInstanceState: Bundle?) {
        super.onCreate(savedInstanceState)
        enableEdgeToEdge()
        setContent { Game() }
    }
}
```

- onCreate se ejecuta al abrir la app (ciclo de vida de la Activity).
- enableEdgeToEdge() dibuja la app hasta los bordes de la pantalla.
- setContent { Game() } indica que la interfaz es la funcion Game.

6. Que cambio respecto a la version anterior

Antes	Ahora
Column/Row con dp fijos y weight	Lienzo 746x720 escalado con una unidad u
Se estiraba en tablet	Mantiene proporcion y se centra
Mucho codigo repetido por elemento	3 ayudas (Pieza, Circulo, Texto): 1 linea por elemento
Cruceta = Text("+"), sin interruptores	Cruceta con lineas, interruptores VOL y 3D
Botones inferiores sin borde propio	Borde negro de 2 px en SELECT/HOME/START

7. Preguntas tipicas de examen

- **Que es un Composable?** Una funcion con @Composable que describe un trozo de UI.
- **Diferencia entre Box, Row y Column?** Box apila, Row es horizontal, Column es vertical.
- **Por que el orden de los Modifier importa?** Se aplican en orden: no es lo mismo padding antes que despues de background.
- **Como evitas que la UI se estire en tablet?** Calculando una unica escala u = min(ancho/ANCHO, alto/ALTO) y multiplicando todas las medidas por ella.
- **Para que sirve BoxWithConstraints?** Para conocer el espacio disponible (maxWidth, maxHeight) y adaptar el diseño.
- **Que es un parametro con valor por defecto?** Un parametro opcional (forma = RectangleShape).
- **Que es una lambda composable?** Un bloque de UI pasado como parametro (contenido), usado para anidar piezas.
- **dp vs sp?** dp para tamaños de elementos, sp para texto.
- **Para que sirve @Preview?** Ver el diseño en Android Studio sin ejecutar la app.